Le patron de conception Commande

Le patron de conception Commande (Command Pattern) est un patron comportemental dont l'objectif principal est de séparer celui qui demande une action de celui qui la réalise réellement. Autrement dit, il permet de découpler l'émetteur d'une requête (par exemple un bouton, un menu, une interface graphique) de l'objet qui sait comment effectuer l'action (un service, un composant métier, un périphérique, etc.).

Pour y parvenir, l'action à exécuter est encapsulée dans un objet appelé *commande*. Cet objet devient une représentation autonome de la requête : il peut être stocké, transmis, exécuté plus tard, annulé ou combiné avec d'autres commandes.

Pourquoi a-t-on besoin du patron Commande ?

Dans de nombreux systèmes logiciels, on souhaite déclencher des actions sans que le déclencheur connaisse les détails de leur implémentation. Sans le patron Commande, l'émetteur d'une action doit appeler directement des méthodes spécifiques sur des objets concrets. Cela entraîne un couplage fort, rendant le système difficile à faire évoluer, à maintenir ou à tester.

Le patron Commande apporte une solution élégante en introduisant une interface commune pour toutes les actions possibles. L'émetteur n'interagit alors qu'avec cette interface, ce qui permet de changer les comportements dynamiquement, d'ajouter de nouvelles actions sans modifier le code existant, et de respecter le principe *Open/Closed*.

Principe de fonctionnement

Le cœur du patron Commande repose sur une idée simple :

Une action est un objet.

Chaque action est représentée par une classe qui implémente une interface commune (par exemple une méthode `execute()`). Lorsque cette méthode est appelée, la commande délègue le travail à un autre objet qui sait réellement comment faire l'opération. Ainsi, l'émetteur de la commande n'a aucune connaissance du fonctionnement interne de l'action.

Les rôles du patron Commande

Le patron Commande est structuré autour de cinq rôles fondamentaux.
Le Command est une interface ou une classe abstraite qui définit les méthodes communes à toutes les commandes, généralement `execute()` et parfois `undo()`.
Le ConcreteCommand est une implémentation concrète de cette interface. Elle encapsule l'appel à un ou plusieurs objets responsables de l'action.
Le Receiver est l'objet qui réalise effectivement le travail. Il contient la logique métier.
Le Invoker est l'objet qui déclenche la commande. Il ne connaît pas le Receiver, seulement la Commande.

Enfin, le Client est responsable de créer les commandes et de configurer les relations entre les commandes, les receivers et les invokers.

Cette séparation claire des responsabilités rend l'architecture plus modulaire et plus flexible.

Le patron Commande présente plusieurs avantages majeurs. Il permet tout d'abord un faible couplage entre les composants du système. Il facilite également l'extensibilité, car de nouvelles commandes peuvent être ajoutées sans modifier le code existant. Il rend possible la gestion de l'historique des actions, ce qui est essentiel pour les fonctionnalités d'annulation (*Undo/Redo*). De plus, il permet de composer des commandes, par exemple sous forme de macros ou de scripts.

Malgré ses nombreux atouts, le patron Commande peut entraîner une augmentation du nombre de classes, car chaque action est représentée par une classe distincte. Dans des systèmes très simples, son utilisation peut sembler excessive. Il est donc surtout pertinent lorsque le nombre d'actions est important ou lorsque la flexibilité et l'évolutivité sont des critères essentiels.

Le patron Commande est largement utilisé dans les interfaces graphiques (boutons, menus, raccourcis clavier), les éditeurs de texte (annuler/refaire), les systèmes de traitement par lots, les files de tâches, les systèmes de macro-commandes, ou encore dans les architectures orientées événements.

Le patron de conception Commande transforme les actions en objets autonomes, ce qui permet de construire des systèmes flexibles, extensibles et faiblement couplés. Il est particulièrement adapté aux situations où les requêtes doivent être paramétrées, enregistrées, annulées ou combinées, tout en conservant une architecture claire et évolutive.

Exercice : Patron de conception Commande

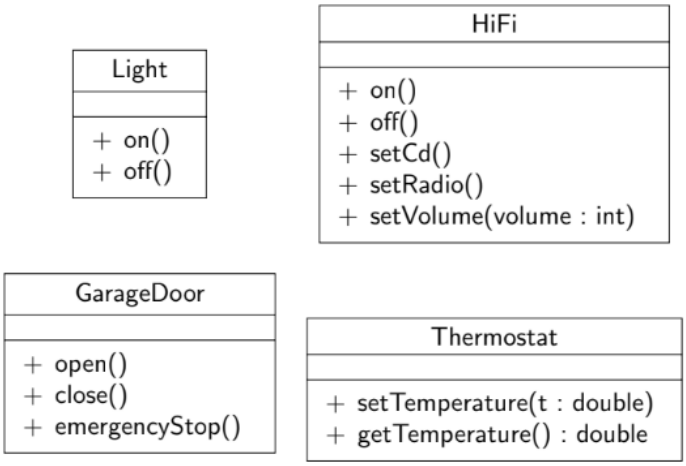
On souhaite développer une application domotique permettant à un utilisateur de contrôler divers équipements de sa maison ou appartement. Par exemple, via cette application, l'utilisateur pourra allumer ou éteindre les lumières, activer la chaîne HiFi, ouvrir ou fermer la porte du garage, gérer l'alarme ou encore régler le thermostat.

L'application sera représentée par une interface graphique simple, composée (Figure) :

- De plusieurs paires de boutons On/Off (pour activer/désactiver les services),
- D'un bouton « Undo » qui permet d'annuler la dernière action réalisée (un seul niveau d'annulation est prévu),
- D'une fonctionnalité permettant à l'utilisateur d'associer librement chaque paire de boutons à un service spécifique.

Le design minimaliste de l'interface s'explique par la volonté de développer à terme un contrôleur physique qui reproduira cette interface (avec seulement des boutons On/Off).

On dispose d'un modèle d'interface correspondant à une classe appelée `RemoteControl`. Cette classe gère l'association entre les boutons et les services. Certains services sont déjà fournis sous forme de classes provenant de divers fournisseurs (illustrées en figure). D'autres services pourront être ajoutés à l'avenir.



Remarque : dans un système plus avancé, l'interface pourrait proposer des contrôles supplémentaires (ex. un curseur pour régler la température du thermostat).

Objectifs de l'exercice

1. Permettre d'associer n'importe quel service à une paire de boutons.
2. Gérer l'annulation d'une commande via le bouton « Undo ».

Étapes de l'étude

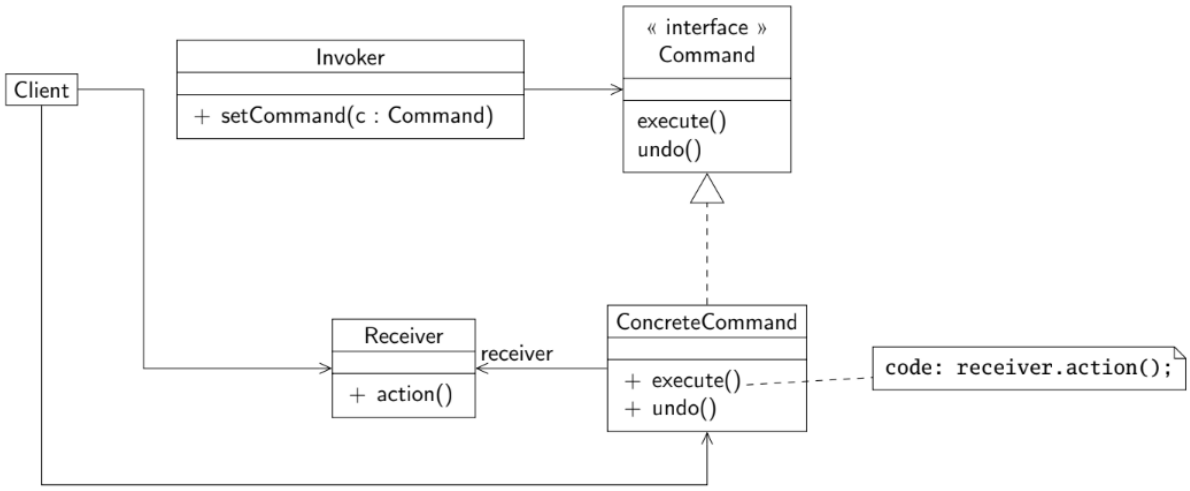
1. Cas simple : Supposons d'abord que les classes représentant les services sont modifiables. Quelle solution simple proposeriez-vous pour lier les services aux boutons ?
2. Cas réel : Maintenant, supposons que ces classes sont fournies sous forme de bytecode (et donc non modifiables).

a) Comment peut-on associer des actions à chaque paire de boutons sans avoir de classe parente commune pour tous les services ?
b) Pour résoudre cela, nous allons utiliser le patron de conception Commande.

Patron de conception Commande

Le patron Commande a pour objectif de découpler l'émetteur d'une requête **(**celui qui déclenche une action) de son récepteur (celui qui exécute réellement l'action**)**. Il encapsule une action (ou un ensemble d'actions) dans un objet, appelé Commande, que l'on peut stocker, exécuter, annuler, ou combiner avec d'autres.

Rôle	Description
Command	Interface ou classe abstraite définissant une méthode `execute()`.
ConcreteCommand	Classe concrète qui implémente `Command` et qui encapsule les appels vers un ou plusieurs objets "réels" (les Receivers).
Receiver	Objet qui connaît les opérations à effectuer. C'est sur lui que l'action est exécutée (ex : lampe, porte, chaîne HiFi...).
Invoker	L'émetteur. C'est lui qui déclenche l'exécution de la commande (ex : l'utilisateur appuie sur un bouton). Il ne connaît pas le Receiver, seulement la Commande.
Client	Crée les commandes, configure les liaisons entre les commandes et les invokers.



Travaux demandés

3. Diagramme de séquence
   - Représentez la séquence d'interactions entre le Client, le Invoker (ex. RemoteControl), le Command et le Receiver.
4. Implémentation de base en Java
   - Écrivez l'interface Command.

- Implémentez une commande LightOnCommand pour allumer une lampe.

5. Classe RemoteControl
   - Implémentez la classe en utilisant le patron Commande.
   - Supposez qu'il y a 5 boutons On/Off. Stockez les commandes dans deux tableaux (onCommands[], offCommands[]).
   - Ajoutez les méthodes suivantes :
     - buttonOnPressed(int i) : simule l'appui sur le i^e bouton On
     - buttonOffPressed(int i) : simule l'appui sur le i^e bouton Off
     - undoButtonPressed() : annule la dernière commande exécutée
   - Assurez-vous qu'une commande soit bien assignée avant de l'exécuter.
6. Proposez une solution simple pour éviter d'avoir à vérifier manuellement qu'une commande est assignée avant son exécution.
7. Cas du thermostat
   - Quelles sont les deux actions possibles (On/Off) sur le thermostat ?
   - Quel problème cela pose-t-il avec ce patron ?
   - Proposez une solution pour gérer ce cas particulier.
8. Commandes macros

   a. Créez une classe MacroCommand permettant de regrouper plusieurs commandes. Proposez un diagramme de classes et codez la classe MacroCommand.

   b. Ajoutez une fonctionnalité d'enregistrement de macros :

   - Ajoutez deux nouveaux boutons : StartRecordingMacro et StopRecordingMacro
   - Spécialisez la classe RemoteControl pour gérer l'enregistrement des commandes exécutées entre ces deux boutons.
   - Ajoutez les méthodes :
     - startRecordingMacroPressed()
     - stopRecordingMacroPressed()